Cost-Efficient Segmentation under a Compute Budget

CSL7360: Computer Vision – Assignment 2

Track B: Fully Convolutional Network

Student: Lokesh Motwani

1 Introduction

This report presents **FastSegUNet**, a fully convolutional network developed from scratch for binary salient-object segmentation. Given a 256×256 image, the model predicts a mask that labels each pixel as foreground or background. The assignment imposes an average single-core CPU inference budget of 50 ms per image and evaluates the trade-off between segmentation quality, runtime and saved parameter size.

The quality score is

$$Q = 0.70\,\mathrm{mIoU} + 0.30\,\mathrm{BF1},$$

where mIoU measures region overlap and Boundary F1 (BF1) measures boundary alignment. The official Cost-Efficient Segmentation score is

$$\mathrm{CES} = \begin{cases} Q\left(1 - 0.20\,\frac{T_{\mathrm{CPU}}}{50\ \mathrm{ms}} - 0.10\,\min\left(1, \frac{S_{\mathrm{disk}}}{4\ \mathrm{MiB}}\right)\right), & T_{\mathrm{CPU}} \le 50\ \mathrm{ms}, \\ 0, & T_{\mathrm{CPU}} > 50\ \mathrm{ms}. \end{cases}$$

Here, T_{CPU} is the mean CPU time per image on the complete DUTS-TE set, and S_{disk} is the saved parameter-file size in bytes. The 4 MiB value is the reference in the capped size-penalty term.

2 Methodology

2.1 FastSegUNet architecture

FastSegUNet is a compact U-Net-style encoder–decoder FCN. It is trained from scratch and uses depthwise-separable convolution blocks to reduce computation. Each `DSConvBlock` applies a 3×3 depthwise convolution followed by a 1×1 pointwise convolution, with batch normalization and ReLU after each convolution.

The final configuration uses `base_ch = 16`. The encoder progressively reduces the spatial resolution, while the decoder upsamples features and concatenates corresponding encoder features through skip connections.

Table 1: Summary of the final FastSegUNet architecture.

Stage	Resolution	Output channels	Operation
Stem	$256 \to 128$	16	Stride-2 convolution and DSConv block
Encoder 2	$128 \to 64$	32	Stride-2 DSConv block followed by DSConv block
Encoder 3	$64 \to 32$	48	Stride-2 DSConv block followed by DSConv block
Encoder 4	$32 \to 16$	64	Stride-2 DSConv block followed by DSConv block
Bottleneck	$16 \to 8$	$96 \to 64$	Downsampling DSConv block and feature-processing block
Decoder 4	$8 \to 16$	48	Bilinear upsampling, skip concatenation and DSConv block
Decoder 3	$16 \to 32$	32	Bilinear upsampling, skip concatenation and DSConv block
Decoder 2	$32 \to 64$	16	Bilinear upsampling, skip concatenation and DSConv block
Decoder 1	$64 \to 128$	16	Bilinear upsampling, skip concatenation and DSConv block
Refinement and head	$128 \to 256$	16; then 1	Full-resolution DSConv refinement and 1×1 prediction head

The network produces one logit per pixel. During inference, logits greater than zero are assigned to the foreground class. The final mask is represented as a 256×256 binary array.

2.2 Training loss

The training objective combines edge-weighted binary cross-entropy and soft Dice loss:

$$\mathcal{L} = 0.5\,\mathcal{L}_{\mathrm{BCE,edge}} + 0.5\,\mathcal{L}_{\mathrm{Dice}}.$$

For the BCE term, pixels in the ground-truth boundary band receive an increased weight. The edge-boost coefficient is 2.0, with pixel weights formed as $1 + 2.0\,b$, where b denotes the boundary-band indicator. The Dice term complements the pixel-wise classification loss by accounting for foreground overlap.

2.3 Inference pipeline and efficiency

The submitted inference path takes a 256×256 BGR image already in memory and returns a 256×256 binary mask. Within this timed path, the image is converted to RGB, converted to a PyTorch tensor, passed through the network, and thresholded at a logit value of zero. Resizing the original image to 256×256 is performed before timing, as specified in the assignment.

The following measures are used to reduce inference overhead:

- **Compact operations:** depthwise-separable convolutions are used throughout the encoder and decoder, with bilinear upsampling and skip concatenation.
- **TorchScript:** the model is traced and frozen before repeated inference. This reduces Python-level model invocation overhead; no assumption is made that every operation is fused into a single operator.
- **Single-thread execution:** numerical libraries are restricted to single-thread mode using environment variables (`OMP_NUM_THREADS`, `MKL_NUM_THREADS`, `OPENBLAS_NUM_THREADS`, and `VECLIB_MAXIMUM_THREADS`). Thread counts in PyTorch and OpenCV are also set to 1.
- **Warm-up:** two inference passes are performed before the timed repetitions.
- **Checkpoint storage:** the saved state dictionary uses FP16 values. The parameters are converted to FP32 when loaded for inference.

The reported DUTS-TE timing is measured one image at a time using `time.process_time()`, after warm-up. It includes the in-memory BGR-to-RGB conversion, tensor preparation, network inference and mask thresholding, but excludes image-file I/O and the resize to 256×256.

3 Training Setup

The model is trained using the DUTS-TR training split. The 10,553 image–mask pairs are shuffled with a fixed random seed of 42 and divided into a 90:10 training–validation split:

- **Training set:** 9,498 images.
- **Validation set:** 1,055 images.
- **DUTS-TE:** 5,019 images, reserved for final evaluation and not used for training, tuning or checkpoint selection.

Training is performed on a Kaggle GPU for 35 epochs, using AdamW with an initial learning rate of 10^{-3} and weight decay of 10^{-4}. The batch size is 32. A cosine-annealing learning-rate schedule is used, with a minimum learning rate of 10^{-5}. The checkpoint with the highest validation Q is retained. Figure 1 shows the training loss and validation metrics across the 35 epochs.

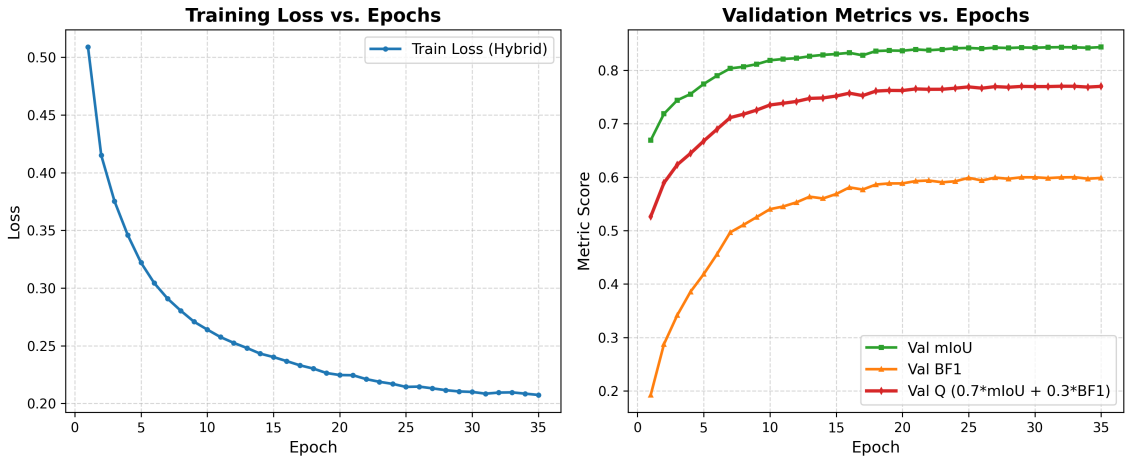


Figure 1: FastSegUNet training progress over 35 epochs on DUTS-TR. Left: hybrid training loss ($0.5\,\mathcal{L}_{\text{BCE,edge}} + 0.5\,\mathcal{L}_{\text{Dice}}$). Right: validation metrics (mIoU, Boundary F1, and Q) on the 1,055-image validation split.

Images are resized to 256×256 using `cv2.INTER_AREA`; masks are resized using `cv2.INTER_NEAREST` and thresholded at 128. The training pipeline applies horizontal flips, occasional vertical flips, and random brightness/contrast changes. The model receives RGB image tensors in the range $[0, 255]$ and performs division by 255 in its forward pass.

The CPU used for the reported inference benchmark is an **AMD Ryzen 5 7520U with Radeon Graphics**. Inference is run on a single CPU thread.

4 Quantitative Results

The metrics below are taken from the final V3 evaluation record. mIoU is computed from the accumulated confusion matrix over a dataset, averaging the IoU for the foreground and background classes. Boundary F1 is computed per image using the assignment's 5×5 boundary matching tolerance, then averaged across the dataset.

Table 2: Segmentation quality, single-core CPU latency, parameter checkpoint size, and CES score across the validation set, official test set (DUTS-TE), and captured-photo set.

Dataset	**Images**	**mIoU**	**BF1**	Q	T_{CPU}	S_{disk}	**CES**
Validation (DUTS-TR)	1,055	0.8433	0.5995	0.7702	27.01 ms	148,255 B	0.6842
DUTS-TE (official)	**5,019**	**0.6883**	**0.4298**	**0.6107**	**26.04 ms**	**148,255 B**	**0.5450**
Captured photos	6	0.8164	0.5198	0.7274	26.04 ms	148,255 B	0.6490

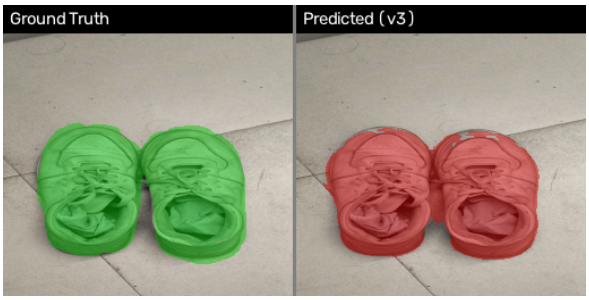
For DUTS-TE, the saved parameter file is 148,255 bytes (approximately 144.8 KiB). The timing is below the 50 ms budget. The score components are:

$$
\begin{aligned}
Q &= 0.70(0.6883) + 0.30(0.4298) \approx 0.6107, \\
P_{\mathrm{time}} &= 0.20 \left(\frac{26.0417}{50} \right) \approx 0.1042, \\
P_{\mathrm{size}} &= 0.10 \left(\frac{148255}{4194304} \right) \approx 0.0035, \\
\mathrm{CES} &= 0.6107(1 - 0.1042 - 0.0035) \approx \mathbf{0.5450}.
\end{aligned}
$$

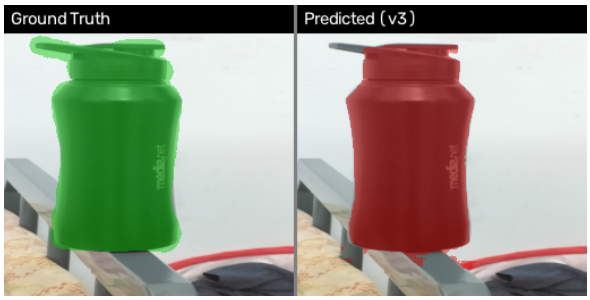
The measured mean CPU time is approximately 48% below the 50 ms limit. The checkpoint occupies approximately 3.5% of the 4 MiB reference size.

5 Qualitative Results on Captured Photos

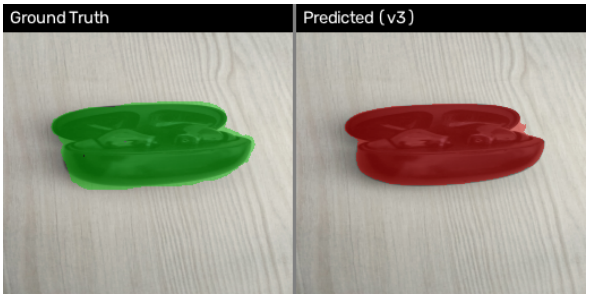
The captured-photo set contains six images: `photo1`, `photo2`, `photo5`, `photo6`, `photo7` and `photo8`. Each image is evaluated against its manually prepared binary mask at 256×256. Figure 2 shows the manually annotated ground truth and V3 prediction as semi-transparent overlays. Green denotes ground truth and red denotes the V3 prediction.



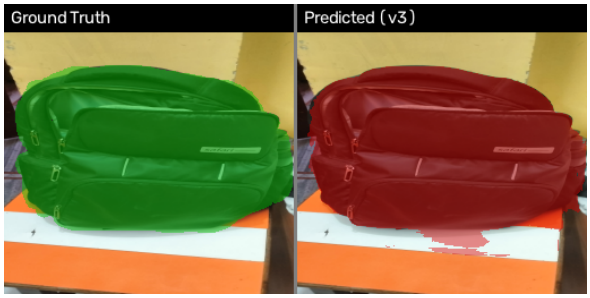
(a) **photo1**: Shoes (mIoU 0.931, BF1 0.549, Q 0.817)



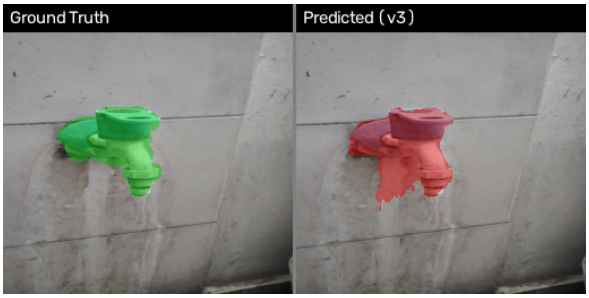
(b) **photo2**: Shaker bottle (mIoU 0.924, BF1 0.513, Q 0.800)



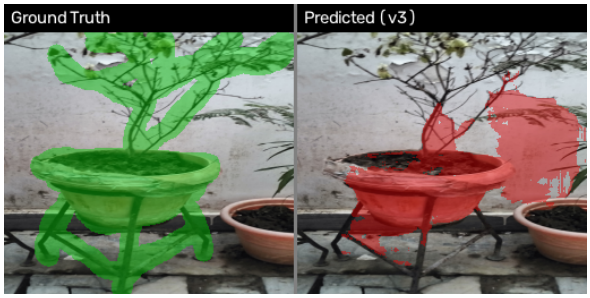
(c) **photo5**: Earbuds case (mIoU 0.948, BF1 0.672, Q 0.865)



(d) **photo6**: Backpack (mIoU 0.928, BF1 0.600, Q 0.830)



(e) **photo7**: Water tap (mIoU 0.855, BF1 0.635, Q 0.789)



(f) **photo8**: Potted plant (mIoU 0.442, BF1 0.150, Q 0.355)

Figure 2: Captured-photo segmentation overlays. Green: ground-truth mask. Red: FastSegUNet (V3) prediction. In each panel, ground truth and prediction are displayed side by side.

The overlays show close agreement for the main object in photos 1, 2, 5 and 6. Photo 7 illustrates excess predicted foreground below the tap, while photo 8 shows a substantial omission of thin branches and parts of the plant support. These examples are discussed further in Section 7.

Scene-condition coverage. The photographs visibly demonstrate at least three of the assignment's requested conditions:

Table 3: Visible scene conditions represented by the captured-photo set.

Condition	Photographic evidence
Indoor and outdoor scenes	Indoor examples include the shoes (photo 1), earbuds case (photo 5) and backpack (photo 6). The potted plant (photo 8) is photographed outdoors.
Near and farther framing	The bottle (photo 2) and backpack (photo 6) are framed closely, while the tap (photo 7) occupies a much smaller part of a wider wall view.
Cluttered background	Photo 8 includes multiple pots, the plant stand, branches and a textured outdoor setting around the main object.

The photographs do not provide sufficiently clear visual evidence to claim a specific tube-light setup or a deliberate partial-occlusion case, so those conditions are not counted toward the coverage statement.

6 Discussion

6.1 V2-to-V3 experiment

V3 changed two architectural choices relative to V2: the channel width was reduced from `base_ch = 24` to 16, while the refinement block was moved back to full 256×256 resolution. The resulting DUTS-TE measurements are:

Table 4: V2 and V3 DUTS-TE comparison.

Metric	V2	V3	Change
mIoU	0.6980	0.6883	−0.0097
Boundary F1	0.4559	0.4298	−0.0261
Q	0.6253	0.6107	−0.0146
T_{CPU}	42.504 ms	26.042 ms	−38.7%
S_{disk}	249,631 B	148,255 B	−40.6%
CES	0.5153	**0.5450**	+0.0297

Although V3 has lower mIoU, BF1 and Q than V2, its lower inference time and smaller checkpoint reduce the cost penalties sufficiently to improve CES from 0.5153 to 0.5450. The intended improvement in boundary quality was not observed on DUTS-TE.

The comparison changes both channel width and refinement resolution at the same time, so it does not isolate the effect of either choice. A plausible explanation is that reducing channel width lowered representational capacity enough to offset any benefit from full-resolution refinement. The result therefore demonstrates a measured quality–efficiency trade-off, rather than proving that one architectural factor is more important in general.

6.2 Validation-to-test performance gap

The validation Q of 0.7702 is higher than the DUTS-TE Q of 0.6107. The same difference is visible in both mIoU and BF1. This indicates that the model performs less strongly on the held-out test set than on the validation split. Differences in sample difficulty, image characteristics or the limited capacity of the compact network are possible explanations; the reported metrics alone do not establish a single cause.

7 Failure Analysis

7.1 Photo 8: substantial under-segmentation

Photo 8 is the weakest captured example, with mIoU 0.4423, BF1 0.1500 and Q 0.3546. Its ground-truth foreground contains 25,507 pixels, while the model predicts 15,089 foreground pixels. The predicted foreground area is therefore approximately 59.2% of the ground-truth area.

The overlay confirms that the prediction retains much of the pot and central trunk but misses a substantial part of the thin upper branches and lower support structure. The branches occupy only a small number of pixels after resizing and repeated spatial downsampling, making them difficult to preserve. The surrounding branches, neighbouring pot and textured outdoor setting also create a visually cluttered background. These are plausible contributing factors rather than experimentally isolated causes.

7.2 Photo 7: over-segmentation

Photo 7 has mIoU 0.8553, BF1 0.6346 and Q 0.7891. The predicted foreground contains 5,152 pixels compared with 4,318 pixels in the ground truth, a net excess of 834 pixels (approximately 19.3% of the ground-truth foreground area). This indicates an over-segmentation tendency in this example.

The overlay shows that the predicted foreground extends below the tap into a region of the surrounding wall. A nearby stain or shadow may share visual characteristics with the tap and contribute to this leakage, although the image alone cannot establish the model's internal reason for the error. The area difference is a net difference; it should not be interpreted as the exact number of false-positive pixels.

8 Conclusion

FastSegUNet obtains an official DUTS-TE CES score of **0.5450**, with mean CPU inference time of **26.04 ms per image** and a saved parameter file of **148,255 bytes**. It remains within the assignment's 50 ms timing budget and has a compact checkpoint. The DUTS-TE results also show that boundary quality remains a limitation, with BF1 lower than mIoU.

The V2-to-V3 comparison illustrates the trade-off between segmentation quality and inference cost: V3's quality score decreased, but the runtime and model-size reductions improved the final CES score. The captured-photo results show generally strong performance on five of the six examples, alongside a substantial under-segmentation failure on photo 8. The six captured-photo overlays and the visible scene-condition evidence provide a qualitative complement to the aggregate metrics.

End of report.